← Danh sách

0/30 câu đã làm · 0/28 câu chấm máy đúng lần đầu

Làm lại từ đầu

CĐ-11 · MLOps và vận hành mô hình

Luyện kiến thức: không tính giờ, chọn xong là biết đúng/sai và xem giải thích. Tiến độ lưu riêng với Luyện đề.

Mã đề **PETER-CD11-1832** · Người làm: Peter: gửi mã này cho mentor khi chữa bài.

Phần A — Nhận biết

Câu 1 · T11-A07 · 3,5 điểm · Điền số · Nhận biết · tình huống

Một GPU thuê giá **2,40 USD/giờ** (giả định) phục vụ ổn định **20 request/giây** khi chạy đầy tải. Chỉ tính tiền thuê GPU, chi phí cho **1.000 request** là bao nhiêu? **Đơn vị: USD, làm tròn 4 chữ số thập phân.**

Nhập số

Kiểm tra

Câu 2 · T11-A01 · 3 điểm · Trắc nghiệm · Nhận biết · tình huống

Dashboard của API chấm điểm tín dụng ghi: **p95 latency = 420 ms** (tính trên mọi request trong 1 giờ qua). Phát biểu nào diễn giải đúng con số này?

- ○ **A.** Chỉ 5% request có độ trễ không vượt quá 420 ms, còn lại chậm hơn
- ○ **B.** Trong 95% thời gian của giờ đó, API phản hồi đúng 420 ms
- ○ **C.** Độ trễ trung bình của 95% request nhanh nhất trong giờ đó là 420 ms
- ○ **D.** Khoảng 95% request trong giờ đó có độ trễ không vượt quá 420 ms

Câu 3 · T11-A05 · 3 điểm · Trắc nghiệm · Nhận biết · tình huống

Bộ lọc spam email mở rộng sang khách hàng doanh nghiệp: tỉ lệ email tiếng Anh trong lưu lượng tăng từ 10% lên 45%. Với một email cho trước, tiêu chí thế nào là spam **không đổi**. Đây là dạng dịch chuyển nào?

- ○ **A.** Concept drift: quan hệ $P(Y \mid X)$ giữa nội dung email và nhãn spam đã đổi

- **B.** Data drift (covariate shift): $P(X)$ đổi còn $P(Y \mid X)$ giữ nguyên
- **C.** Label shift: tỉ lệ spam $P(Y)$ đổi còn $P(X \mid Y)$ giữ nguyên
- **D.** Training-serving skew: pipeline phục vụ tính đặc trưng khác pipeline huấn luyện

Câu 4 · T11-A03 · 3 điểm · Trắc nghiệm · Nhận biết

Khi dùng **DVC** để quản lý phiên bản dữ liệu huấn luyện, thứ được commit vào Git thường là gì?

- **A.** File `.dvc` / `dvc.lock` chứa hash nội dung, trỏ tới dữ liệu nằm ở remote storage
- **B.** Câu truy vấn SQL lấy dữ liệu; chạy lại truy vấn sẽ cho đúng dữ liệu cũ
- **C.** Chỉ file trọng số model; dữ liệu không cần quản lý vì có thể tải lại từ nguồn
- **D.** Toàn bộ file dữ liệu, để Git tự nén và so sánh khác biệt giữa các phiên bản

Câu 5 · T11-A02 · 3 điểm · Trắc nghiệm · Nhận biết

Tính năng **dynamic batching** của một inference server (ví dụ NVIDIA Triton) làm gì?

- **A.** Lưu kết quả của các request giống hệt nhau để trả lại mà không cần chạy mô hình
- **B.** Gom các request đến gần nhau trong một cửa sổ chờ ngắn thành một batch để tăng throughput GPU
- **C.** Chia một request lớn thành nhiều batch nhỏ chạy song song trên nhiều GPU để giảm độ trễ của chính request đó
- **D.** Tự điều chỉnh batch size khi huấn luyện để loss hội tụ nhanh hơn

Câu 6 · T11-A06 · 3 điểm · Trắc nghiệm · Nhận biết

Triển khai kiểu **shadow** (shadow deployment) cho một model mới nghĩa là gì?

- **A.** Dựng môi trường mới song song, chuyển toàn bộ 100% lưu lượng sang một lần, giữ nguyên môi trường cũ để quay lại khi cần
- **B.** Model mới phục vụ thật một phần nhỏ người dùng, ví dụ 5%, rồi tăng dần nếu ổn

- ○ **C.** Model mới nhận bản sao request thật và chỉ ghi log; người dùng vẫn nhận kết quả của model cũ
- ○ **D.** Hai model phục vụ hai nhóm người dùng ngẫu nhiên để so sánh chỉ số kinh doanh

Câu 7 · T11-A04 · 3 điểm · Trắc nghiệm · Nhận biết

Bài báo *Hidden Technical Debt in Machine Learning Systems* (Sculley và cộng sự, NeurIPS 2015) nêu nguyên tắc **CACE** (*Changing Anything Changes Everything*). Nguyên tắc này cảnh báo điều gì?

- ○ **A.** Nên huấn luyện lại toàn bộ mô hình mỗi khi có dữ liệu mới để tránh nợ kỹ thuật
- ○ **B.** Các đầu vào bị "rối" vào nhau: đổi một đặc trưng có thể làm thay đổi hành vi của cả mô hình
- ○ **C.** Mỗi lần đổi code phải đổi file cấu hình tương ứng, nếu không pipeline sẽ lỗi
- ○ **D.** Thay một mô hình chỉ ảnh hưởng đầu ra của chính nó, nên có thể kiểm thử và cập nhật từng phần riêng rẽ như module phần mềm

Câu 8 · T11-A08 · 3 điểm · Trắc nghiệm · Nhận biết

Muốn build lại Docker image sau 6 tháng vẫn ra đúng cùng base image, dòng `FROM` nào phù hợp nhất?

- ○ **A.** `FROM python:3.11-slim` (tag đã cố định phiên bản)
- ○ **B.** `FROM python:latest` (luôn lấy bản mới nhất)
- ○ **C.** `FROM python:3` kèm `RUN pip install --upgrade pip`
- ○ **D.** `FROM python:3.11-slim@sha256:<digest>`

Phần B — Thông hiểu

Câu 9 · T11-B02 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Chatbot RAG tăng ngữ cảnh truy xuất từ khoảng 1.000 lên 8.000 token mỗi prompt. Đo thấy **TTFT** tăng từ 0,3 s lên 1,6 s, còn tốc độ sinh sau token đầu (token/s) gần như không đổi. Giải thích nào hợp lý nhất?

- ○ **A.** Prefill phải xử lý toàn bộ prompt trước token đầu; decode từng token ít phụ thuộc độ dài prompt
- ○ **B.** KV cache bị tắt khi prompt dài nên mỗi token sinh ra đều phải tính lại từ đầu
- ○ **C.** Gói tin prompt lớn làm mạng chậm: 8.000 token cần thêm khoảng 1,3 s để truyền từ client tới server suy luận
- ○ **D.** Prompt dài khiến mô hình phải sinh nhiều token output hơn nên token đầu tiên ra chậm

Câu 10 · T11-B06 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Canary 2% của model mới chạy 20 phút, nhận **150 request và không có lỗi nào**. Model cũ có tỉ lệ lỗi 0,4%. Kết luận nào đúng?

- ○ **A.** Chưa đủ, vì canary phải chạy ít nhất 24 giờ dù lưu lượng nhiều hay ít
- ○ **B.** Chưa đủ: 0 lỗi trên 150 request cho cận trên 95% của tỉ lệ lỗi khoảng $3/150 = 2\%$
- ○ **C.** Đủ bằng chứng, vì cỡ mẫu 150 lớn hơn 30 nên theo định lý giới hạn trung tâm, ước lượng tỉ lệ lỗi đã đáng tin
- ○ **D.** Model mới tốt hơn rõ rệt vì tỉ lệ lỗi quan sát là 0% so với 0,4%

Câu 11 · T11-B05 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Kiểm toán yêu cầu tái lập model v3 đã huấn luyện 4 tháng trước. Model registry có commit code, siêu tham số và tên bảng `transactions`, nhưng bảng này được cập nhật và ghi đè hằng ngày. Cần bổ sung gì vào quy trình để lần sau tái lập được?

- ○ **A.** Lưu file trọng số v3 thật cẩn thận ở nhiều nơi; tái lập nghĩa là tải lại đúng file trọng số đó khi kiểm toán hỏi
- ○ **B.** Gắn mỗi phiên bản model với snapshot dữ liệu bất biến (hash DVC hoặc time-travel) trong metadata registry
- ○ **C.** Ghi lại số dòng của bảng lúc huấn luyện để kiểm tra dữ liệu chưa thay đổi
- ○ **D.** Lưu thêm random seed; cùng seed, code và siêu tham số thì kết quả sẽ giống hệt

Câu 12 · T11-B04 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Nhóm sửa prompt qua 35 vòng; mỗi vòng chạy **cùng một golden set 150 mẫu** và giữ bản có điểm cao nhất. Bản cuối đạt 91% (ngưỡng cổng CI là 90%). Sau release, chấm 1.000 mẫu lấy ngẫu nhiên từ log mới thì chỉ được 84%. Nguyên nhân chính và cách sửa cổng eval là gì?

- ○ **A.** Ngưỡng 90% quá thấp; chỉ cần nâng ngưỡng cổng CI lên 95% trên cùng golden set đó là ngăn được hiện tượng này
- ○ **B.** Golden set bị dùng lặp lại để chọn prompt nên điểm bị quá khớp; cần thêm holdout ẩn và làm mới tập từ log
- ○ **C.** LLM có tính ngẫu nhiên; chạy lại eval sau release vài lần nữa thì điểm sẽ quay về khoảng 91% như trong CI
- ○ **D.** Golden set quá nhỏ; nhân bản mỗi mẫu 5 lần thành 750 mẫu để điểm ổn định và đáng tin cậy hơn hẳn

Câu 13 · T11-B09 · 3 điểm · Trắc nghiệm · Thông hiểu

Dockerfile sau mất 6 phút để build lại mỗi khi sửa **một dòng** trong `serve.py`, vì `pip` cài lại toàn bộ thư viện:

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "serve.py"]
```

Sửa thế nào là đúng?

- ○ **A.** Gộp mọi lệnh thành một `RUN` duy nhất để giảm số lớp của image và tăng tốc build
- ○ **B.** Đổi base image sang bản `python:3.11` đầy đủ cho có sẵn nhiều thư viện
- ○ **C.** Thêm `--no-cache-dir` vào lệnh `pip install` để build nhanh hơn
- ○ **D.** Chép `requirements.txt` và chạy `pip install` trước, sau đó mới `COPY . .`

Câu 14 · T11-B10 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Khi ra model v5, nhóm đổi định nghĩa đặc trưng `so_giao_dich` **tại chỗ** từ "số giao dịch 30 ngày" thành "số giao dịch 7 ngày" (giữ nguyên tên cột). v5 gặp sự cố nên được rollback về v4 trong registry, nhưng v4 cũng cho dự đoán sai nhiều dù file model v4 không đổi. Nguyên nhân và bài học đúng là gì?

- ○ **A.** v4 nhận đặc trưng theo định nghĩa mới; cần version hoá đặc trưng và giữ bản cũ đến khi hết khả năng rollback
- ○ **B.** v4 đã cũ nên bị concept drift; nên huấn luyện lại v4 trên dữ liệu mới nhất thay vì cố rollback về một model đã lỗi thời
- ○ **C.** File trọng số v4 trong registry đã hỏng; cần lưu kèm checksum cho mỗi file model
- ○ **D.** Rollback phải chuyển dần như canary; chuyển 100% về v4 một lần gây quá tải

Câu 15 · T11-B07 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Mô hình dự đoán vỡ nợ: nhãn thật (khách có vỡ nợ hay không) chỉ biết sau **90 ngày**. Trong 90 ngày đầu sau deploy, nên giám sát gì để phát hiện sớm vấn đề về chất lượng?

- ○ **A.** Dùng chính dự đoán của mô hình làm nhãn tạm để tính accuracy hằng ngày
- ○ **B.** AUC tính hằng ngày trên các request mới, vì AUC là chỉ số chuẩn và ổn định nhất cho bài toán chấm điểm vỡ nợ
- ○ **C.** Drift của đặc trưng đầu vào và của phân phối điểm dự đoán, cùng các kiểm tra chất lượng dữ liệu
- ○ **D.** Chỉ độ trễ và tỉ lệ lỗi HTTP, vì chất lượng mô hình chỉ đo được sau 90 ngày

Câu 16 · T11-B01 · 3,5 điểm · Điền số · Thông hiểu · tình huống

TTFT (ms) của 24 request tới chatbot trong một phút:

312, 287, 455, 301, 298, 1210, 333, 276, 290, 341, 305, 318, 289, 702, 296, 310, 284, 327, 299, 515, 302, 293, 281, 640

Tính **p95 theo phương pháp nearest-rank**: sắp xếp tăng dần, p95 là phần tử thứ $\lceil 0{,}95 \cdot n \rceil$ (đếm từ 1). **Đơn vị: ms, nhập số nguyên.**

Nhập số | Kiểm tra

Câu 17 · T11-B03 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Một tính năng gọi API LLM, mỗi request trung bình **1.500 token input** và **300 token output**. Giá giả định: input 0,50 USD/1 triệu token, output 2,00 USD/1 triệu token. Chi phí API cho **1.000 request** là bao nhiêu?

- ○ **A.** 0,00135 USD
- ○ **B.** 1,35 USD
- ○ **C.** 2,25 USD
- ○ **D.** 0,75 USD

Câu 18 · T11-B08 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Trên cùng một GPU thuê theo giờ, tăng batch size tối đa từ 1 lên 16: throughput tăng từ **50 lên 200 request/s**, p95 tăng từ 60 lên 140 ms (vẫn trong SLO 200 ms). Chi phí GPU cho mỗi 1.000 request thay đổi thế nào?

- ○ **A.** Tăng 16 lần, vì mỗi lượt chạy GPU giờ phải xử lý 16 request
- ○ **B.** Tăng khoảng 2,3 lần, theo đúng tỉ lệ tăng p95 từ 60 lên 140 ms, vì mỗi request chiếm GPU lâu hơn
- ○ **C.** Không đổi, vì vẫn thuê một GPU với cùng đơn giá
- ○ **D.** Giảm còn 1/4, vì giá mỗi giờ không đổi mà số request mỗi giờ tăng 4 lần

Phần C — Vận dụng

Câu 19 · T11-C03 · 3,5 điểm · Điền số · Vận dụng · tình huống

Cần phục vụ một model **34 tỷ tham số** trên GPU 24 GB. Trọng số lượng tử hóa **INT4** (4 bit/tham số), và cứ mỗi **nhóm 128 tham số** lưu thêm một hệ số scale **FP16** (16 bit). Tính bộ nhớ chỉ riêng cho trọng số. **Đơn vị: GB (1 GB = 10^9 byte), làm tròn 2 chữ số thập phân.**

Nhập số | Kiểm tra

Câu 20 · T11-C08 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Cụm GPU phục vụ trợ lý nội bộ: giờ cao điểm 08:00 đến 22:00 (14 giờ) cần **8 GPU**, 10 giờ còn lại chỉ cần **1 GPU**. Hiện đang chạy cố định 8 GPU suốt 24 giờ. Mỗi GPU mới bật cần khoảng 3 phút để tải model. Nhận định nào đúng khi chuyển sang autoscaling?

- ○ **A.** Tiết kiệm khoảng 57% GPU-giờ và không ảnh hưởng độ trễ, vì autoscaler phản ứng gần như tức thì khi tải tăng

- ○ **B.** Tiết kiệm khoảng 36% GPU-giờ, nhưng cần scale-out trước giờ cao điểm do cold start 3 phút
- ○ **C.** Tiết kiệm 87,5% GPU-giờ vì ban đêm chỉ cần 1/8 số GPU
- ○ **D.** Không tiết kiệm được, vì vẫn cần đủ 8 GPU vào giờ cao điểm

Câu 21 · T11-C07 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Dịch vụ có tải ổn định **50 request/s** suốt ngày. Hai loại GPU (giá giả định, tính tiền theo giờ cho cả GPU, không chia nhỏ):

GPU	Giá	Throughput tối đa trong SLO
X	1,20 USD/giờ	30 request/s
Y	3,60 USD/giờ	120 request/s

Phương án nào đủ công suất với chi phí mỗi giờ thấp nhất?

- ○ **A.** 2 GPU X: 2,40 USD/giờ
- ○ **B.** 1 GPU Y: 3,60 USD/giờ
- ○ **C.** 1 GPU Y: chỉ 1,50 USD/giờ
- ○ **D.** 1 GPU X: 1,20 USD/giờ

Câu 22 · T11-C02 · 3,5 điểm · Điền số · Vận dụng

Đo trên một GPU: thời gian một lượt chạy mô hình với batch b request là $t(b) = 20 + 5b$ (ms). Server luôn gom đủ batch $b = 16$ (giả sử lưu lượng đủ lớn để GPU chạy liên tục). Throughput đạt bao nhiêu? **Đơn vị: request/giây, nhập số nguyên.**

Nhập số | Kiểm tra

Câu 23 · T11-C04 · 3,5 điểm · Điền số · Vận dụng · tình huống

API tóm tắt văn bản đặt cache phía trước model. Mọi request đều tra cache mất **15 ms**; nếu hit thì trả luôn, nếu miss thì gọi model mất thêm **900 ms**. Tỉ lệ cache hit đo được là **35%**. Độ trễ trung bình mỗi request là bao nhiêu? **Đơn vị: ms, làm tròn 1 chữ số thập phân.**

Nhập số

Kiểm tra

Câu 24 · T11-C06 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Pipeline CD tự động promote model lên production nếu đạt cổng eval offline. Model mới đạt AUC offline 0,91 nhưng chạy thật sai nhiều. Điều tra cho thấy bước tiền xử lý ở dịch vụ phục vụ được viết lại bằng Java và không chuyển chữ về chữ thường như code Python lúc huấn luyện. Nên bổ sung kiểm thử nào vào CI?

- ○ **A.** Test đối chiếu: cùng request mẫu qua tiền xử lý của huấn luyện và của phục vụ phải ra đặc trưng khớp nhau
- ○ **B.** Nâng ngưỡng AUC offline của cổng eval từ 0,90 lên 0,93
- ○ **C.** Chạy eval offline với 5 seed khác nhau rồi lấy trung bình AUC, giúp cổng eval ổn định hơn và giảm rủi ro promote nhầm model
- ○ **D.** Unit test bảo đảm hàm huấn luyện chạy hết mà không ném ngoại lệ

Câu 25 · T11-C05 · 3,5 điểm · Điền số · Vận dụng · tình huống

Giám sát đặc trưng `so_tien_giao_dich` chia 5 khoảng. Tỉ lệ khi huấn luyện (expected): **[0,10; 0,20; 0,40; 0,20; 0,10]**. Tỉ lệ tuần này (actual): **[0,05; 0,15; 0,35; 0,25; 0,20]**. Tính PSI $= \sum_i (a_i - e_i) \ln \frac{a_i}{e_i}$ (logarit tự nhiên). **Làm tròn 3 chữ số thập phân.**

Nhập số

Kiểm tra

Câu 26 · T11-C09 · 3 điểm · Trắc nghiệm · Vận dụng

Huấn luyện lại một mạng nơ-ron với **cùng code, cùng dữ liệu, cùng seed** trên GPU, nhưng accuracy hai lần chạy lệch nhau 0,3 điểm. Cách xử lý nào đúng hướng?

- ○ **A.** Đổi sang seed lớn hơn, ví dụ 10 chữ số, để giảm dao động giữa các lần chạy
- ○ **B.** Kết luận dữ liệu bị rò rỉ giữa train và test, cần chia lại tập dữ liệu
- ○ **C.** Rebuild Docker image trước mỗi lần chạy để môi trường luôn mới nhất và giống nhau giữa các lần chạy
- ○ **D.** Bật chế độ thuật toán tất định của framework và đặt seed cho mọi nguồn ngẫu nhiên

Câu 27 · T11-C01 · 3,5 điểm · Điền số · Vận dụng · tình huống

Dịch vụ LLM nhận trung bình **12 request/giây**, thời gian xử lý trung bình mỗi request (từ lúc vào đến lúc xong) là **2,5 giây**. Mỗi replica xử lý đồng thời tối đa **8 request**, và nhóm muốn giữ mức dùng slot không quá **80%**. Dùng định luật Little, cần tối thiểu bao nhiêu replica? **Nhập số nguyên.**

Nhập số

Kiểm tra

Phần D — Vận dụng cao

Câu 28 · T11-D02 · 6 điểm · Tự luận · Vận dụng cao · tình huống

Bạn phụ trách đưa **model xếp hạng gợi ý v7** thay cho v6 trên ứng dụng thương mại điện tử: khoảng **2 triệu request/ngày**, SLO hiện tại p95 ≤ 250 ms, tỉ lệ lỗi 5xx của v6 là 0,2%, chỉ số kinh doanh chính là tỉ lệ thêm vào giỏ (add-to-cart, hiện 6,0%). Offline, v7 tốt hơn v6 về NDCG@10 nhưng dùng thêm 2 đặc trưng mới từ feature store.

Hãy viết **kế hoạch rollout canary** gồm:

1. Các giai đoạn (tỉ lệ lưu lượng, thời gian tối thiểu hoặc số request tối thiểu mỗi giai đoạn, cách chọn người dùng vào canary).
2. Các chỉ số theo dõi và **tiêu chí rollback định lượng** (ngưỡng cụ thể, so với nhóm đối chứng v6 cùng thời điểm).
3. Cơ chế rollback (làm sao quay về trong vài phút, xử lý 2 đặc trưng mới) và việc cần làm sau rollback.

Viết câu trả lời…

Xem bài mẫu & rubric

Viết thử trước rồi hãy mở.

Câu 29 · T11-D03 · 6 điểm · Tự luận · Vận dụng cao · tình huống

Nhóm vận hành **chatbot hỏi đáp chính sách bảo hành** dùng RAG. Mỗi pull request có thể đổi prompt, đổi model LLM, đổi cấu hình retriever hoặc cập nhật tài liệu. Hiện chưa có kiểm tra tự động nào; hai lần gần đây release làm sai câu trả lời về thời hạn bảo hành mà không ai phát hiện trước.

Hãy **thiết kế cổng eval (eval gate) trong CI** cho dự án này, nêu rõ:

1. Dữ liệu eval: thành phần, cách gán nhãn, cách quản lý phiên bản, chống quá khớp.
2. Chỉ số và luật pass/fail định lượng (tuyệt đối và so với baseline của nhánh chính), gồm cả phân khúc quan trọng.
3. Cách xử lý tính ngẫu nhiên của LLM, giới hạn thời gian/chi phí chạy và những gì báo cáo trên PR.

Viết câu trả lời…

Xem bài mẫu & rubric Viết thử trước rồi hãy mở.

Câu 30 · T11-D01 · 3,5 điểm · Điền số · Vận dụng cao · tình huống

Một tính năng đang gọi API LLM: mỗi request 1.200 token input (0,40 USD/1 triệu) và 250 token output (1,60 USD/1 triệu). Lưu lượng **8 triệu request/tháng**. Phương án tự host: 2 GPU giá **1,80 USD/giờ/GPU**, chạy 730 giờ/tháng, cộng **1.500 USD/tháng** công vận hành; chi phí chuyển đổi một lần **12.000 USD** (mọi giá đều là giả định). Giả sử tự host đủ công suất và chất lượng tương đương. Sau bao nhiêu tháng thì hoàn vốn khoản chuyển đổi? **Đơn vị: tháng, làm tròn 1 chữ số thập phân.**

Nhập số Kiểm tra